

Introduction to Computing (CS 101)

For Loops

T. Venkatesh & John Jose



Department of Computer Science & Engineering
Indian Institute of Technology Guwahati

Structured Programming

- All programs can be written in terms of only three control structures
 - Sequence, selection and repetition
- **The sequence structure**
 - Unless otherwise directed, the statements are executed in the order in which they are written.
- **The selection structure**
 - Used to choose among alternative courses of action.
- **The repetition structure**
 - Allows an action to be repeated while some condition remains true.

Selection: the if-else statement

```
□ if ( condition )
```

```
□ {
```

```
□ statement(s)
```

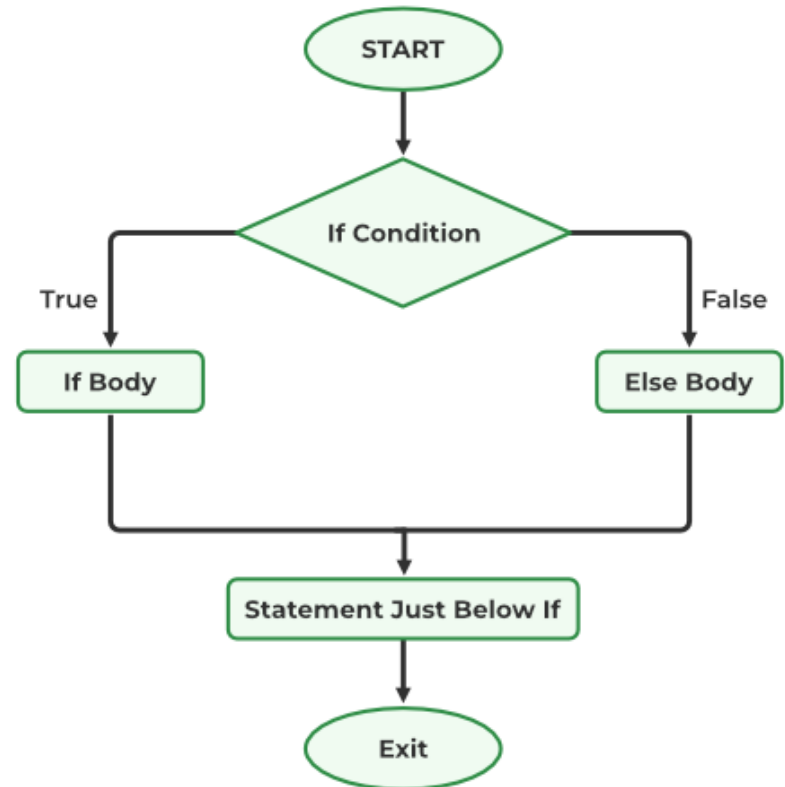
```
□ }
```

```
□ else
```

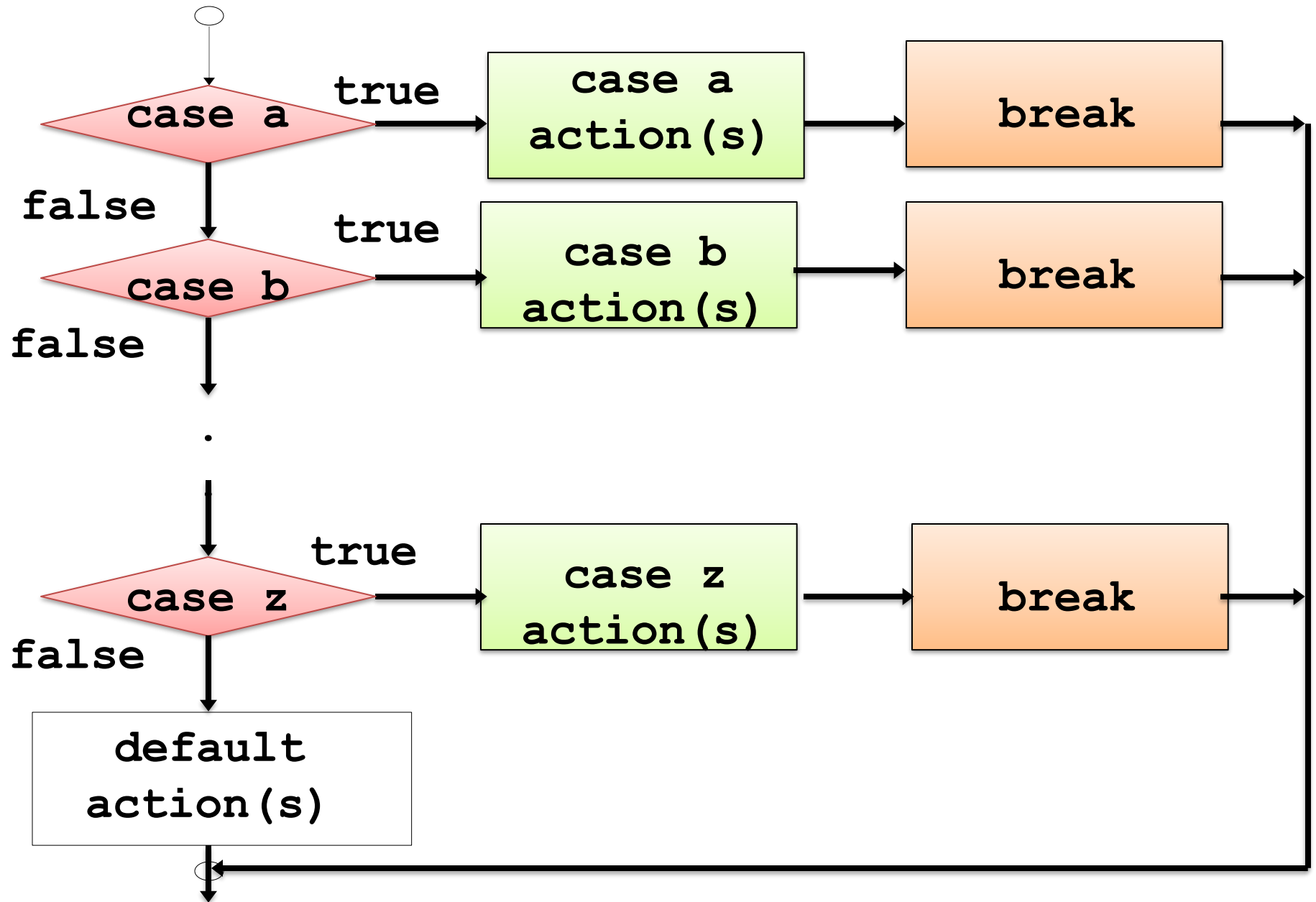
```
□ {
```

```
□ statement(s)
```

```
□ }
```



Flowchart of Switch Statement



The for loop

- The **while** loop is pretty general.
 - Anything that can be done **using repetition** can be done **with a while loop**
- Because counting is so common
 - There is a specialized construct
 - Called : **for loop**.
- **for loop**
 - Makes it easy to set up a counting loop

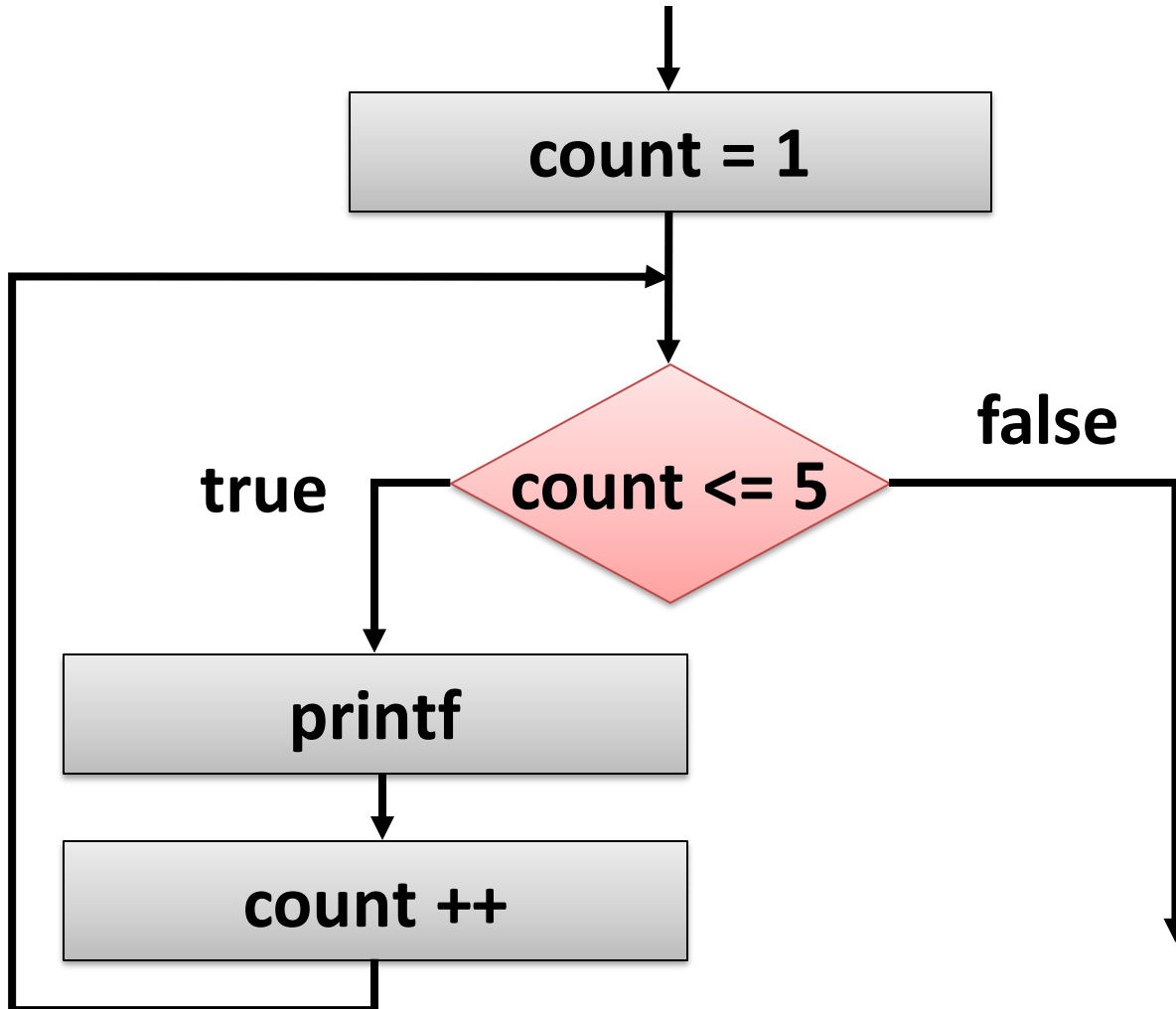
For loop: Three parts

- Three parts to a for loop (just like the while):
 - Set the initial value for the counter
 - Set the condition for the counter
 - Set how the counter changes each time through the loop

```
for ( count=1; count<=5; count++ ) {  
    statement;  
}
```

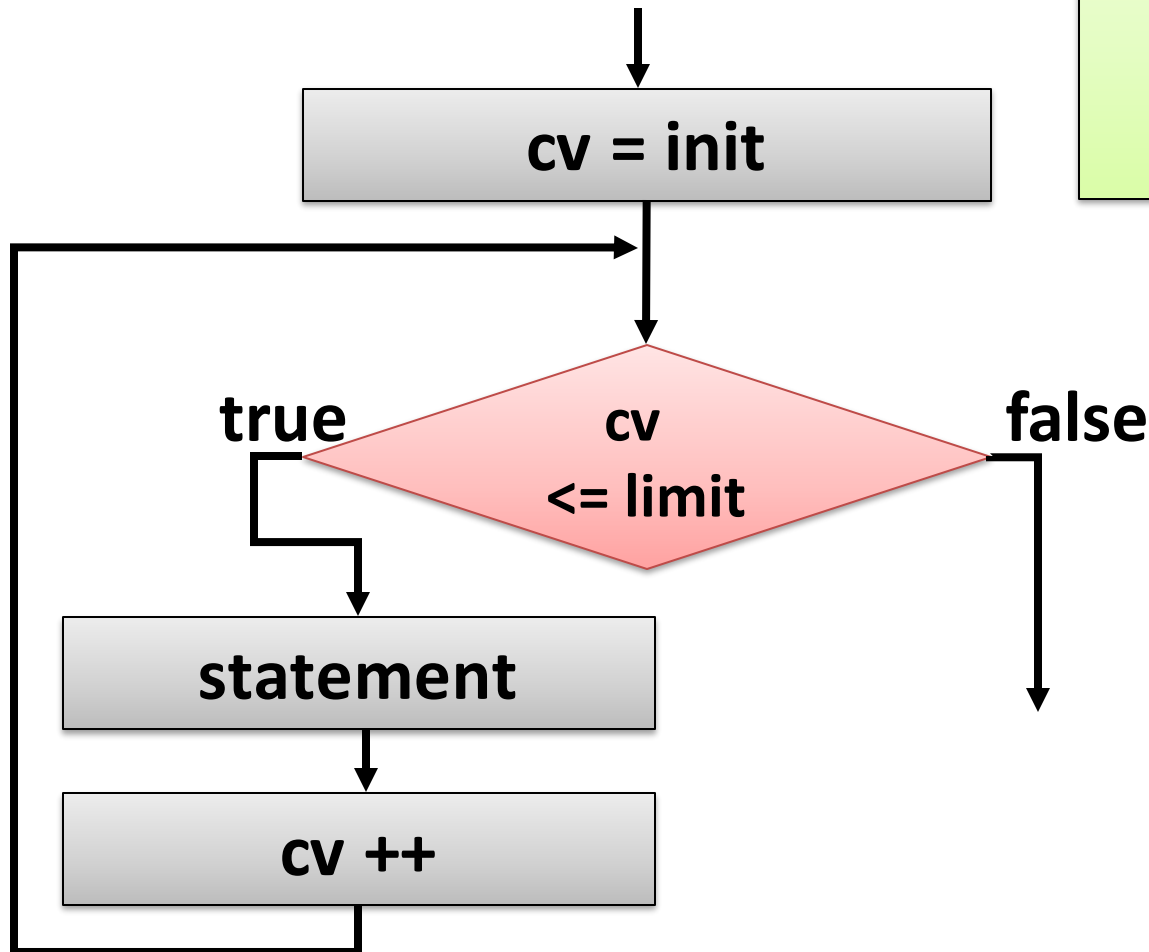
For Loop: Example

```
for (count=1; count<=5; count++)  
    printf("count=%d\n", count);
```



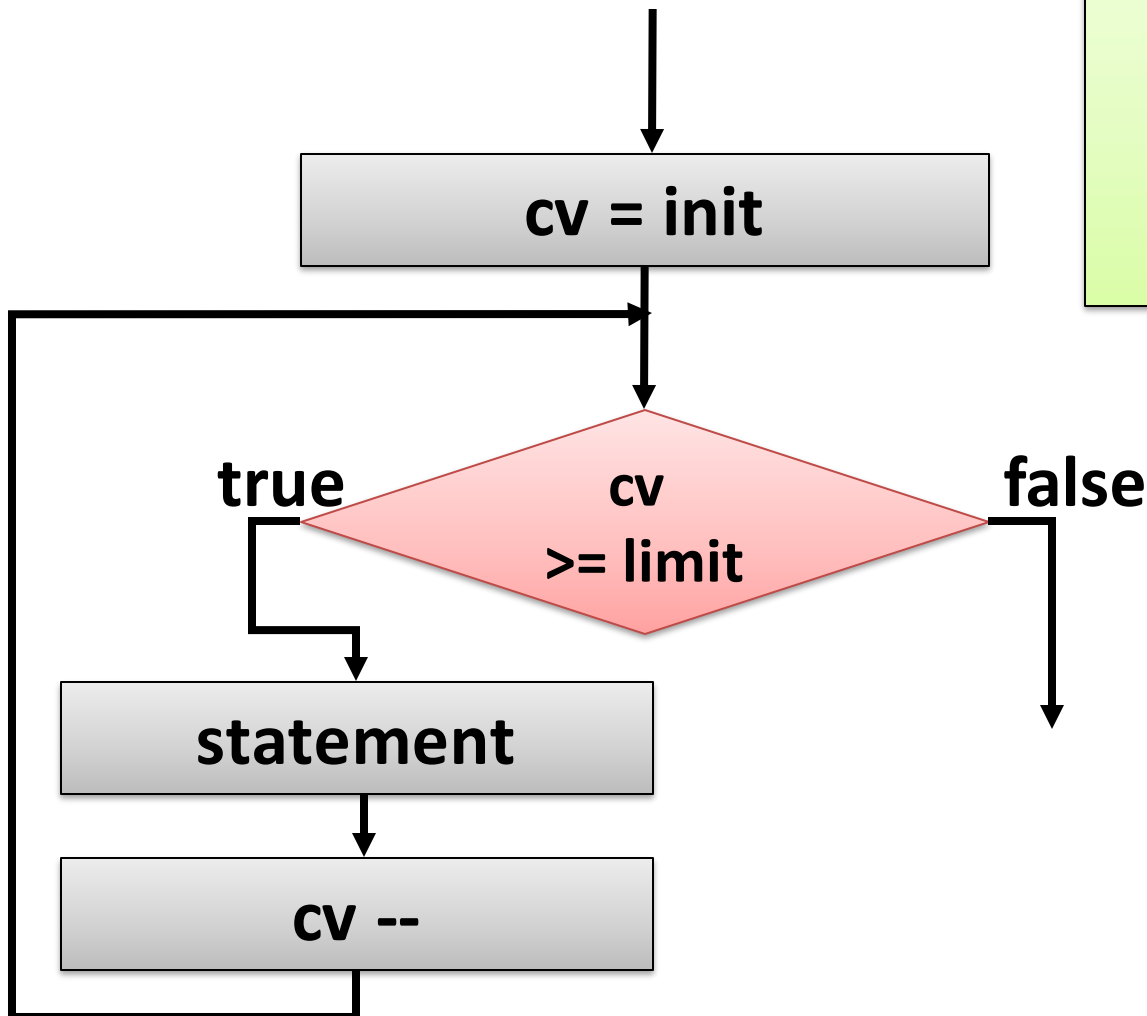
For loop: Ascending for

```
for (cv=init; cv  
<=limit; cv++) {  
    statement;  
}
```



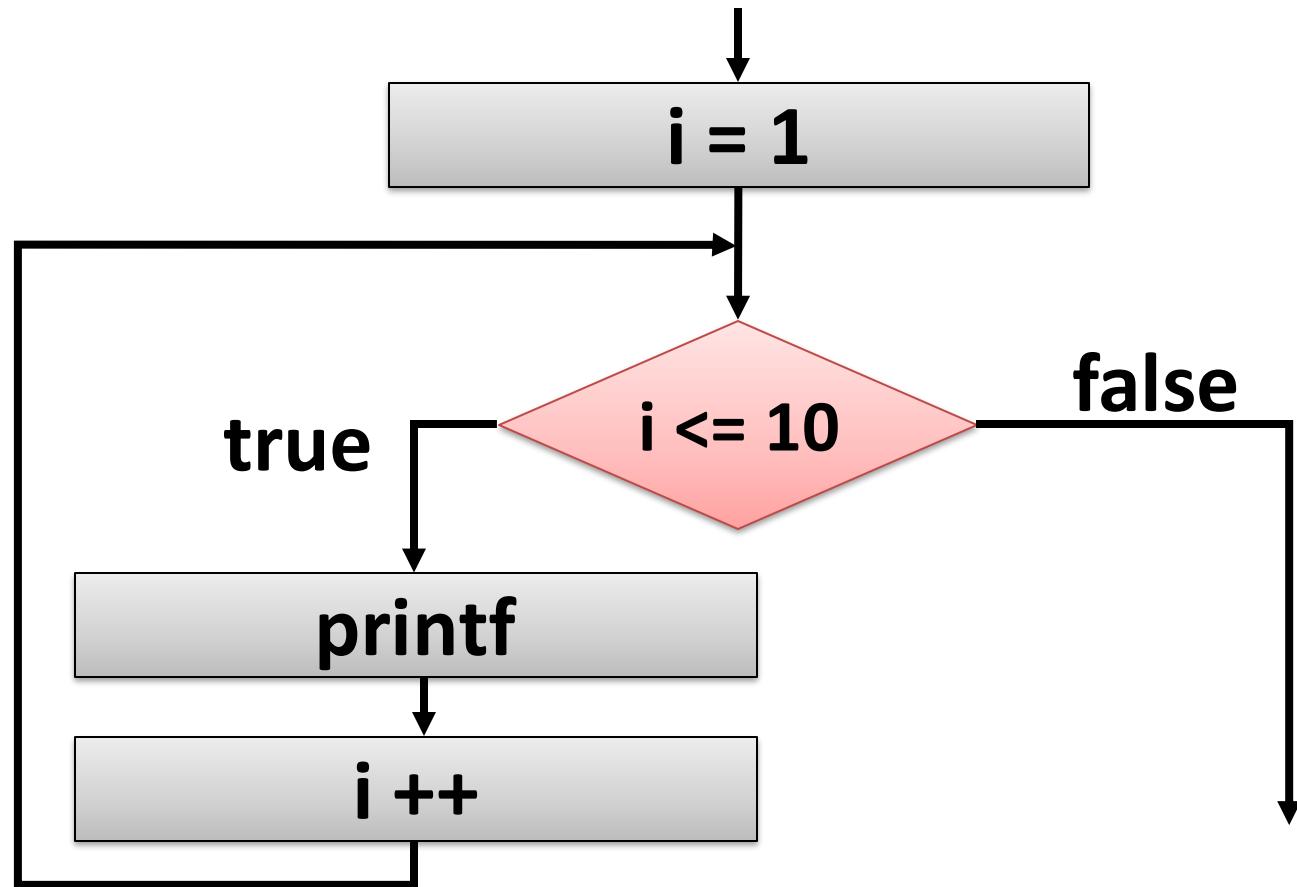
For Loop : Descending for

```
for (cv=init; cv  
>=limit; cv--) {  
    statement;  
}
```



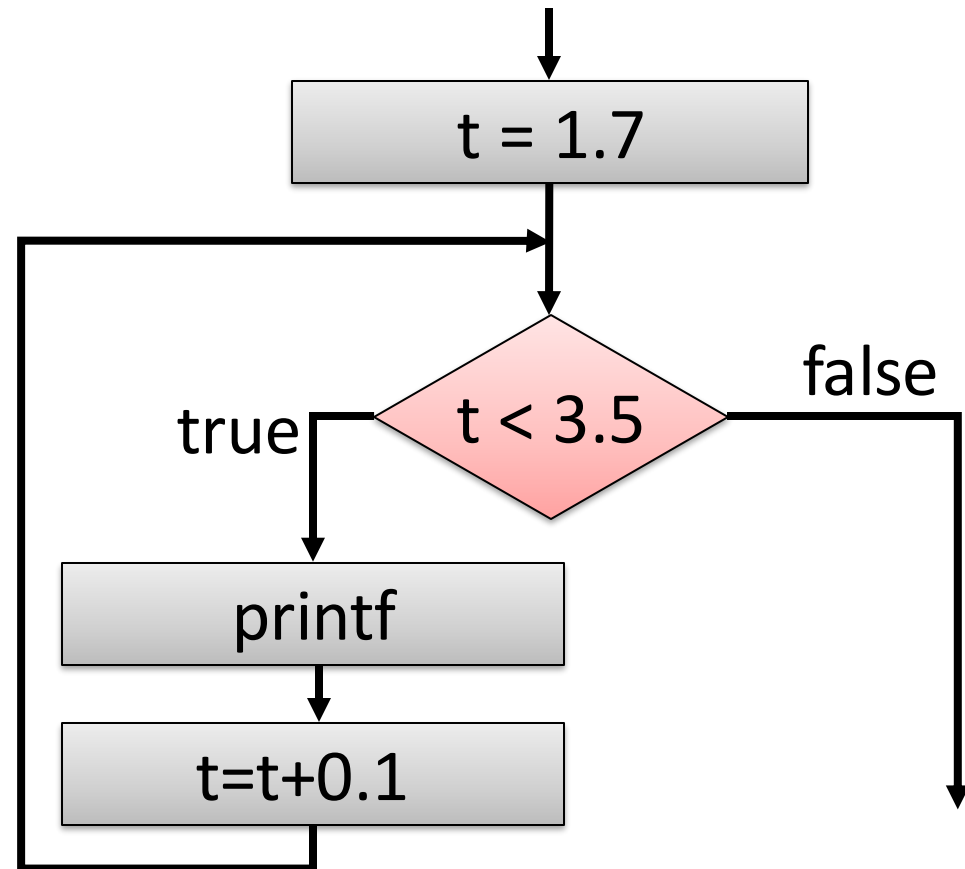
For loop : Examples

```
for (i=1; i<=10; i++) {  
    printf("%d\n", i);  
}
```



For loop Examples : Float ctrl variable

```
for (t=1.7; t<3.5; t=t+0.1) {  
    printf("%f\n", t);  
}
```



For Loop: Possible condition error

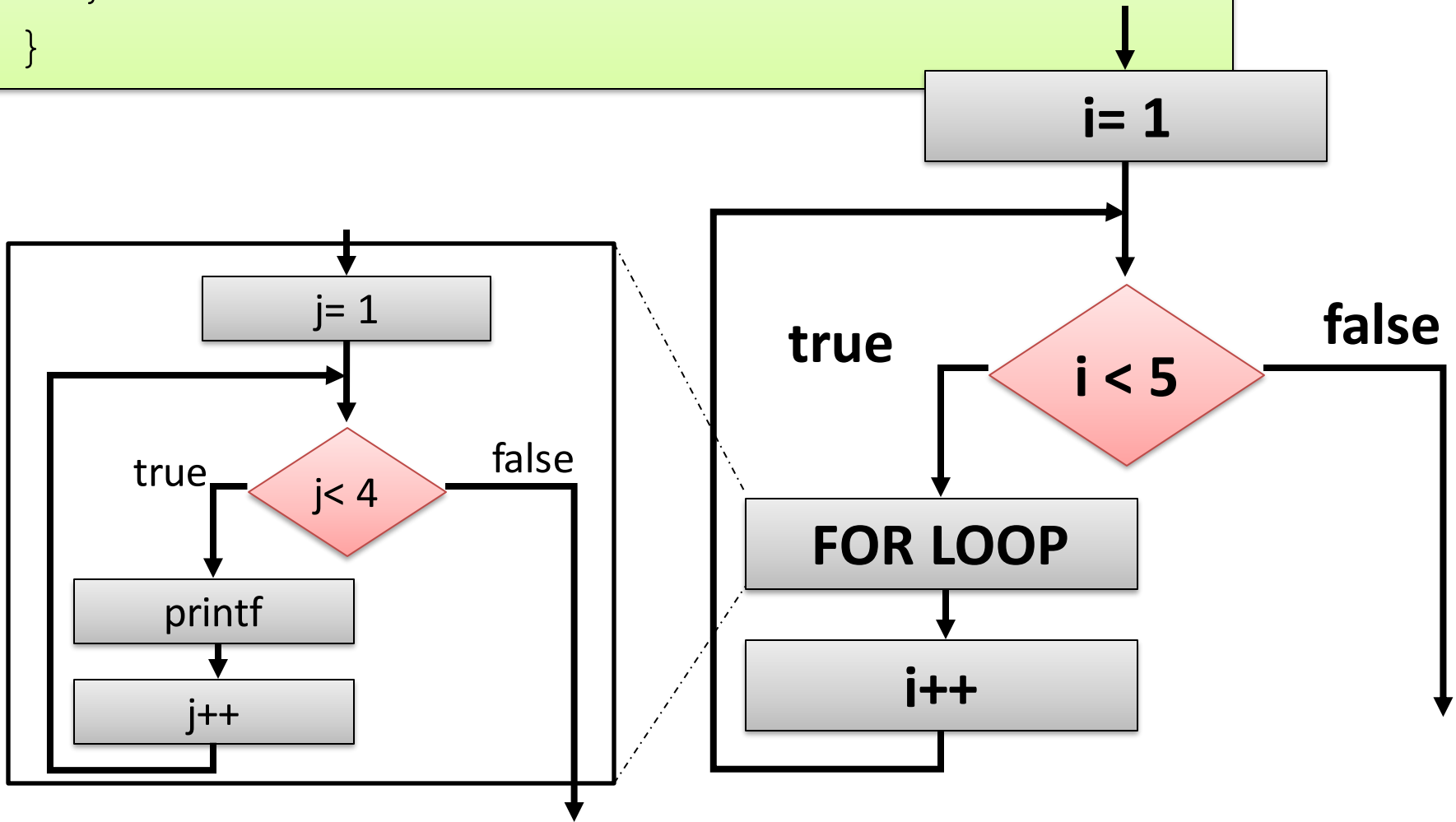
- It is easy to get a for loop to be “one off” of the number you want.
- Be careful of the combination of **initial value** and $<$ vs. $\leq$

–for (i=0; i<10; i++)	10 values: 0 to 9
–for (i=0; i<=10; i++)	11 values: 0 to 10
–for (i=1; i<10; i++)	9 values: 1 to 9
–for (i=1; i<=10; i++)	10 values: 1 to 10

- Counting from 0, with $<$, is a good combination and good for invariants as well
[As array indexing in C start with 0 : will be discussed later]

Nested For loop : Examples

```
for (i=1; i<5; i++) {  
    for (j=1; j<4; j++) {  
        printf("%d * %d = %d\n", i, j, i*j);  
    }  
}
```



Equivalence of top-tested while loop and for loop

- The following loop

```
for (x=init; x<=limit; x++) {  
    statement_list  
}
```

- Is equivalent to



```
x=init;  
while (x<=limit) {  
    statement_list;  
    x++;  
}
```

For-ever or infinite loop

- Used for event driven case
- Mostly event break the infinite loop using break statement : **coming out of the loop**

```
while (1) {  
    /* Loop until value is valid */  
}
```

```
for (;;) {  
    /* Loop without testing */  
}
```

For-ever or infinite loop

- Used for event driven case
- Mostly event break the infinite loop using break statement

```
while(1) {  
    scanf("%c", &c);  
    if(c=='e' || c=='E') {  
        printf("\nEnter the required  
                character e or E\n");  
        break; // coming out of the loop  
    }  
    printf("%c", c);  
}
```

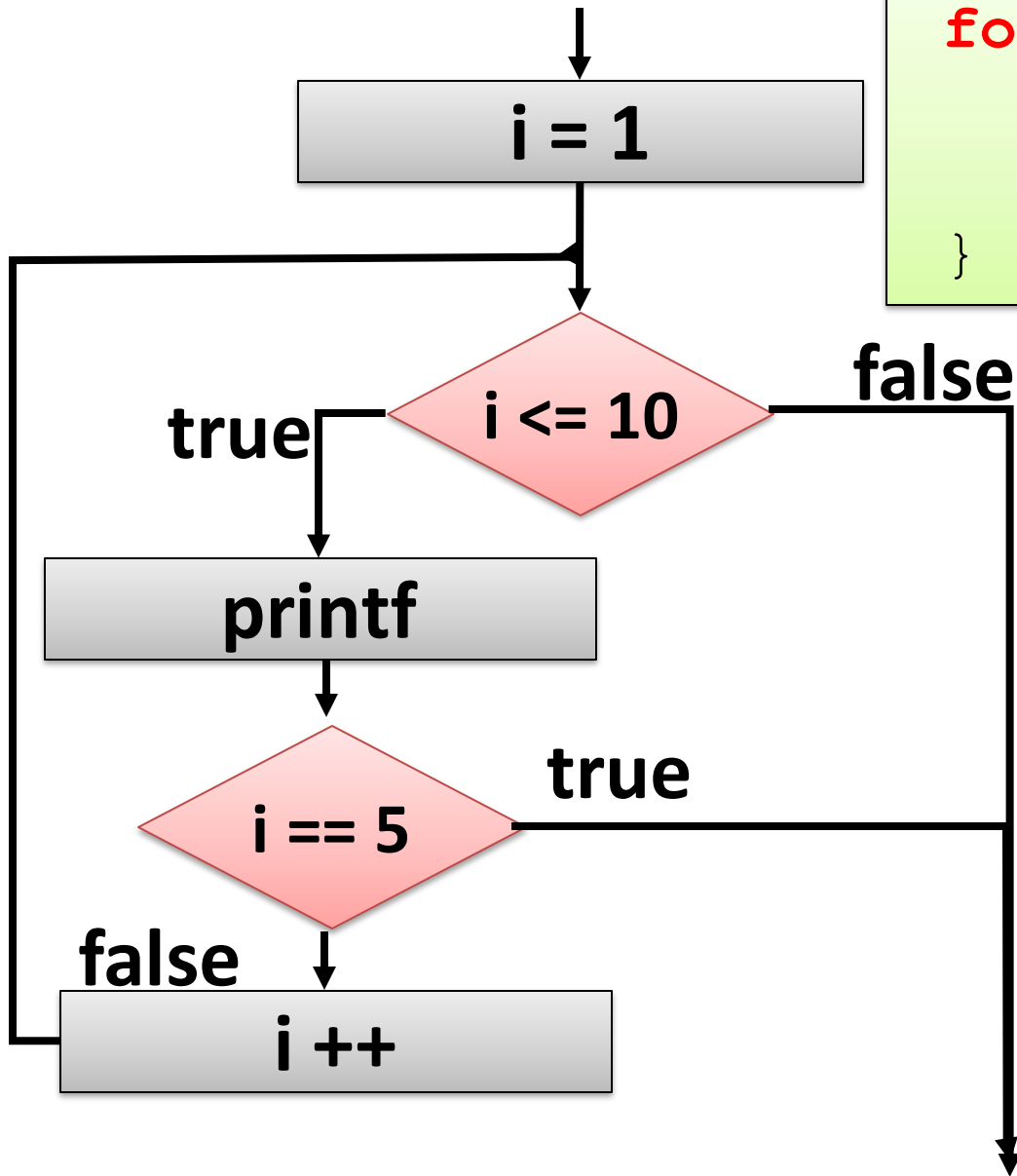

For-ever or infinite loop

- Used for event driven case
- Mostly event break the infinite loop using **break** statement

```
for (;;) {  
    scanf ("%c", &c) ;  
    if (c== 'e' || c== 'E') {  
        printf ("\nEnter the required  
                character e or E\n");  
        break; // coming out of the loop  
    }  
    printf ("%c", c) ;  
}
```

Finite for loop with : break

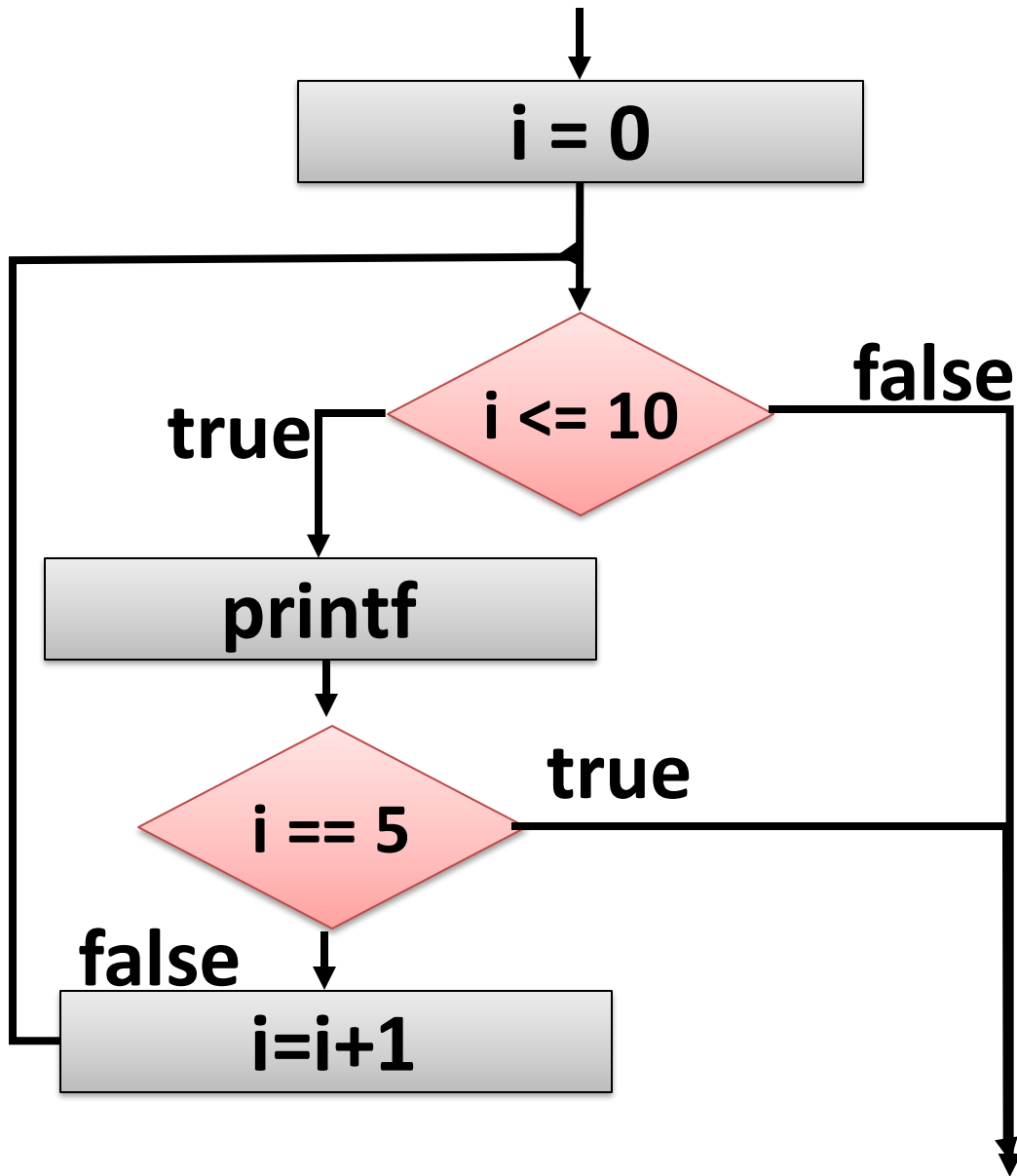
```
for (i=1; i<=10; i++) {  
    printf("%d, ", i);  
    if (i==5) break;  
}
```



1, 2, 3, 4, 5,

// coming out of the loop

Finite while loop with : break

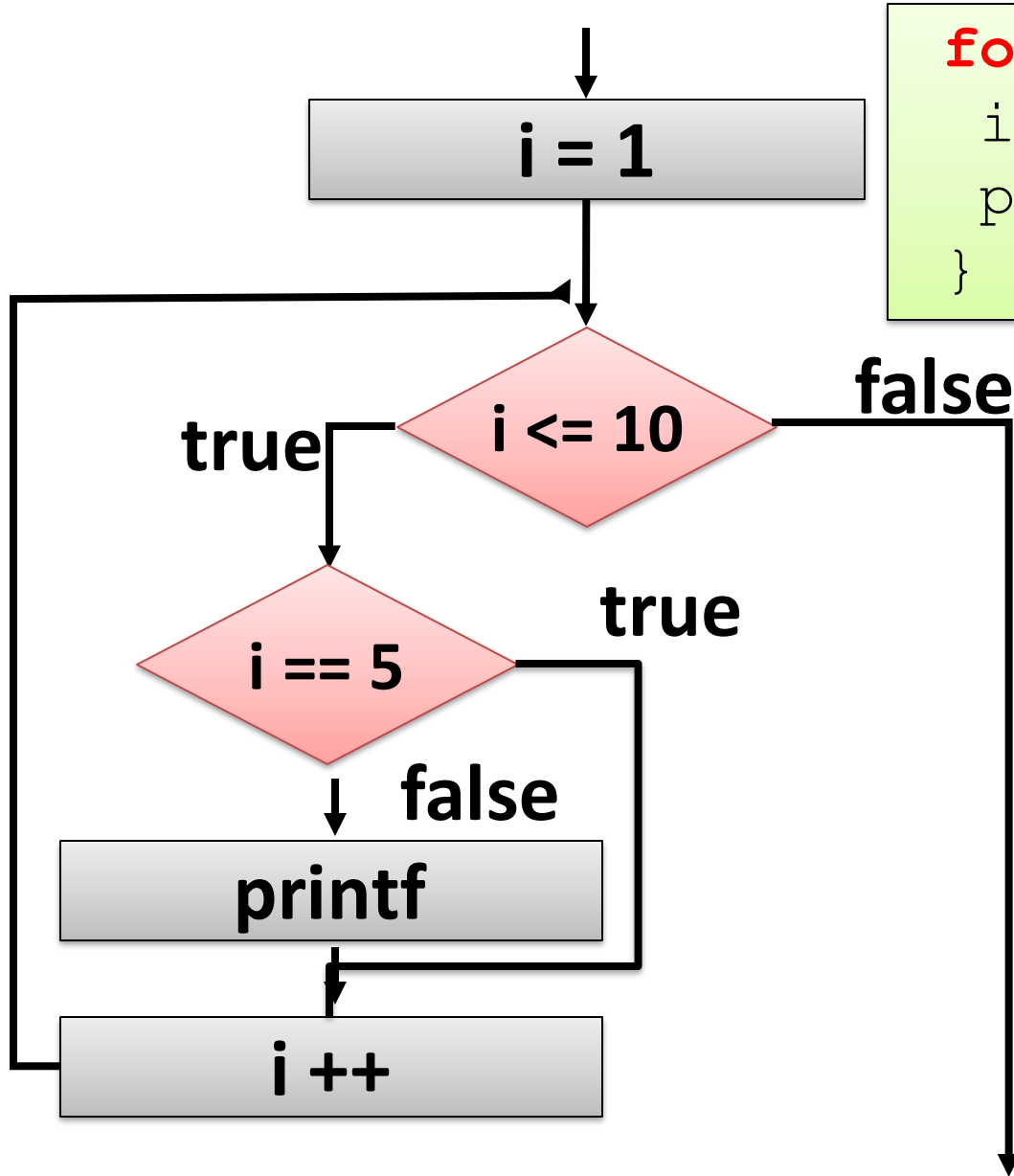


```
i=0;  
while (i<=10) {  
    printf("%d, ", i);  
    if (i==5) break;  
    i=i+1;  
}
```

0, 1, 2, 3, 4, 5,

// coming out of the loop

Finite for loop with : continue

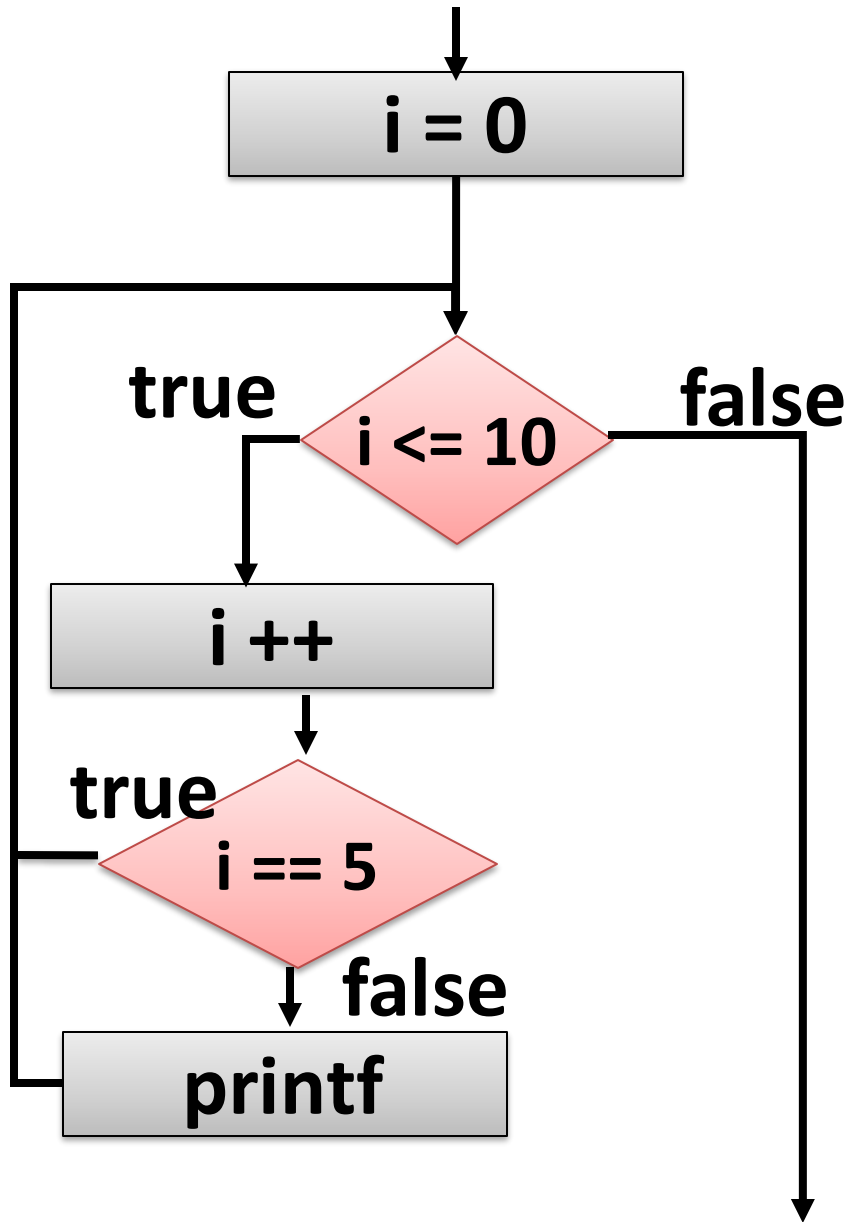


```
for (i=1; i<=10; i++) {  
    if (i==5) continue;  
    printf("%d\n", i);  
}
```

1, 2, 3, 4, 6, 7, 8, 9, 10,

*// Skip a case if
condition satisfied*

Finite while loop with : continue



```
i=0;  
While (i<=10) {  
    i++;  
    if (i==5) continue;  
    printf("%d, ", i);  
}
```

0, 1, 2, 3, 4, 6, 7, 8, 9, 10,

*// Skip a case if
condition satisfied*

Bottom-tested Equivalence

- Bottom-tested do loop (posttest)

```
do {  
    statement;  
} while (condition);
```

- Similar to bottom-tested forever loop

```
for (;;) {  
    statement_list;  
    if (!condition) break;  
}
```

Practice Program

Write a Program using for loops for printing a pattern mentioned below.

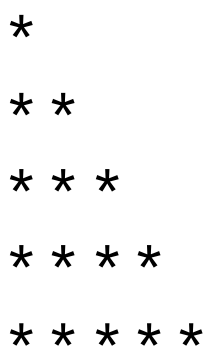
```
*  
* *  
* * *  
* * * *  
* * * * *
```

The number of rows will be defined by user input.

Practice Program

Pattern Printing using “for” loop

```
int main() {  
    int rows, i, j;  
    // Input the number of rows from the user  
    printf("Enter the number of rows: ");  
    scanf("%d", &rows);  
    // Loop to print the star pattern  
    for(i=0, i<rows, i++) {  
        for(j=0, j<=i, j++) {  
            printf("* ");  
        }  
        printf("\n");  
    }  
    return 0;  
}
```



Thanks